Week 7: Create a Simple Java Containerized Application using Docker

Choose an application:

Choose a simple application that you want to containerize. Forexample, a Python script or any Java program

Here we use simple Java program and containerize it using docker

1 Create an application folder

2. Open the Visual Studio Code and select Open Folder from File menu

3. Now select the application folder that is created and open

4. From the explorer bar, click on the application folder

5. Now click on NewFile button to create a file

Now ,Create a simple Java program as below

```
public class Test
{
public static void main(String[] args)
{
int a=20,b=30,c;
c=a*b;
System.out.println("Hello world");
System.out.println("value of c is "+c);
}
}
```

Save it as with .java extension

Now again create another file and name the file as Dockerfile

Create a Dockerfile for the above Java program

In the Dockerfile, **specify the base image, copy the application into thecontainer, and specify the command to run the application**

Dockerfile

FROM openjdk # uses the base image –openjdk with java installed

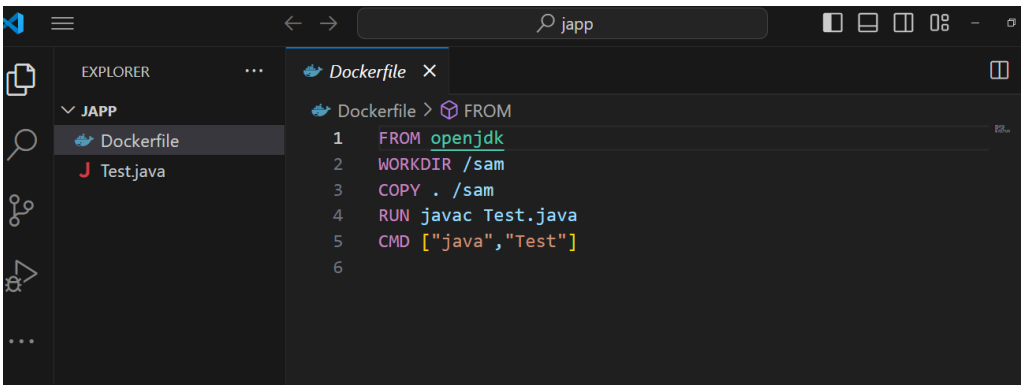
WORKDIR /sam# Set the working directory

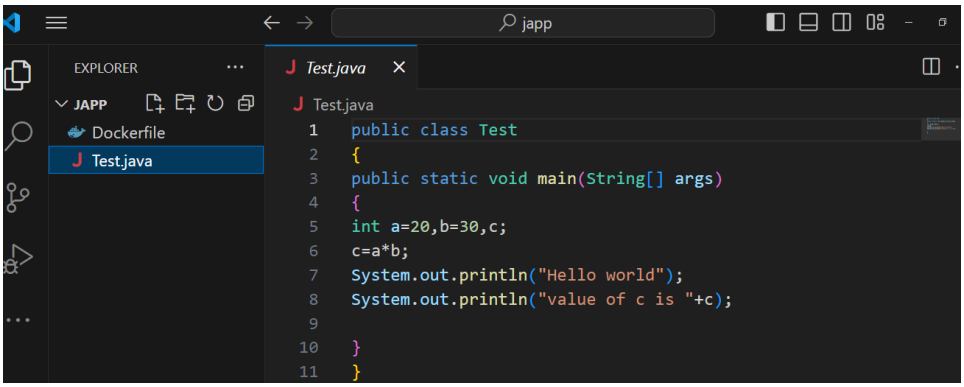
COPY. /sam# Copies all the files to working directory

RUN javac Test.java# Compile the Java program

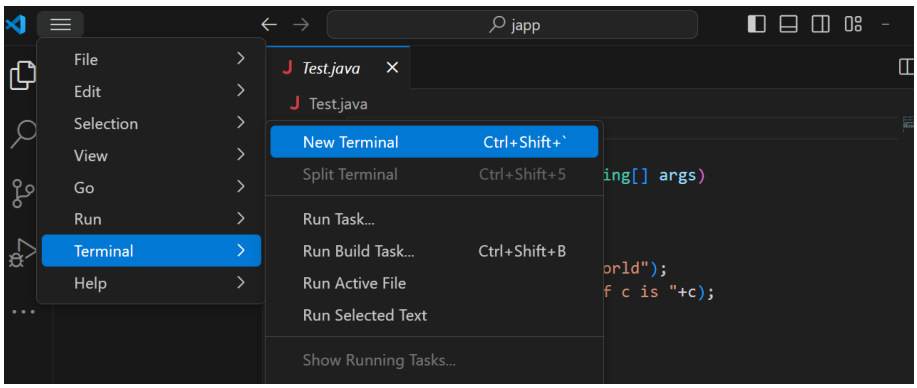
CMD ["java", "Test"]# Set the default command to run the Java program

Save the Java file and Docker file

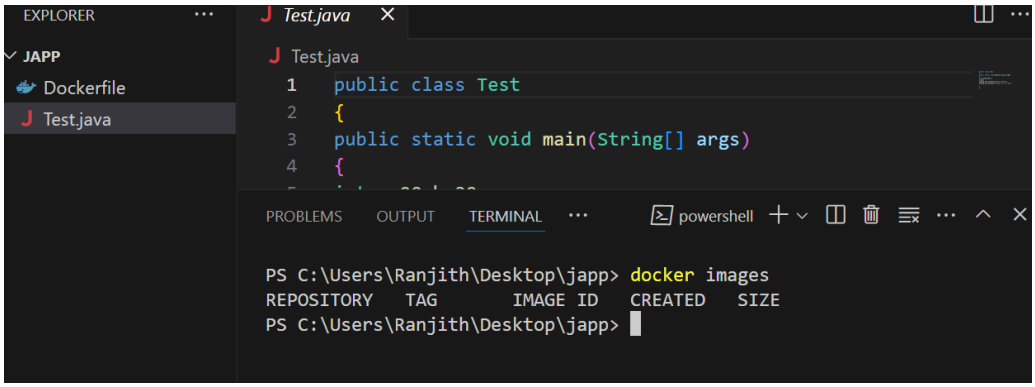




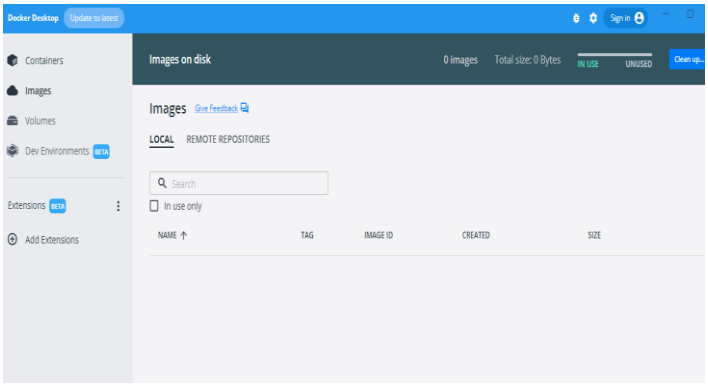
Start the New Terminal from Visual studio



Type the command to list all the images ::::**docker images**



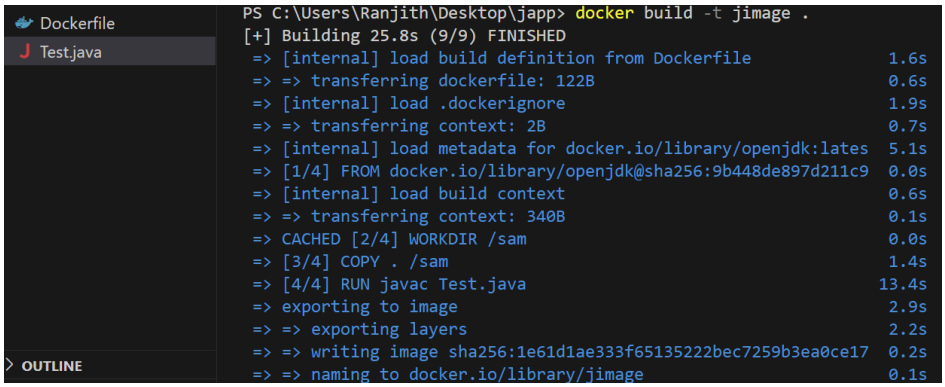
Start the Dockerdesktop



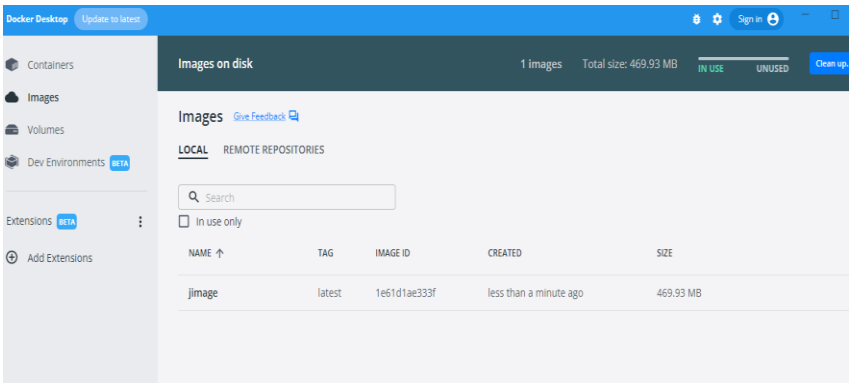
Build the Docker image using the **docker build** command. This command builds a new Docker image using the Dockerfile and tags theimage with the name

Command :docker build -t "any name for your docker image" .

Example :**docker build -t helloworld .**



View the image created in the docker



Run the Docker container:

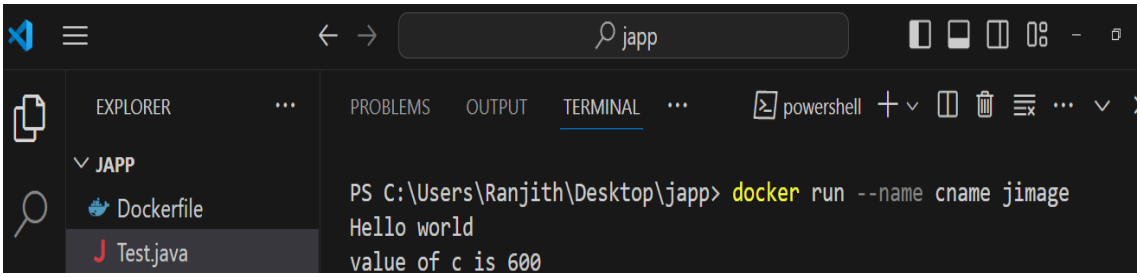
Run the following command to start a new container based on the image:

$ docker run --name mycontainermyimage

This command starts a new container named "mycontainer" based on the

"myimage" image and runs the Java code inside the container.

Example

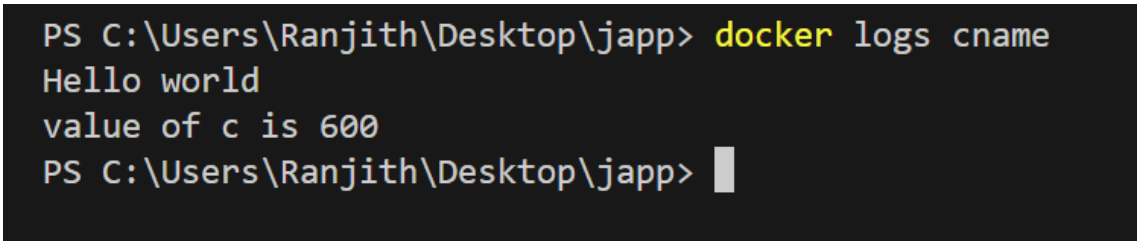


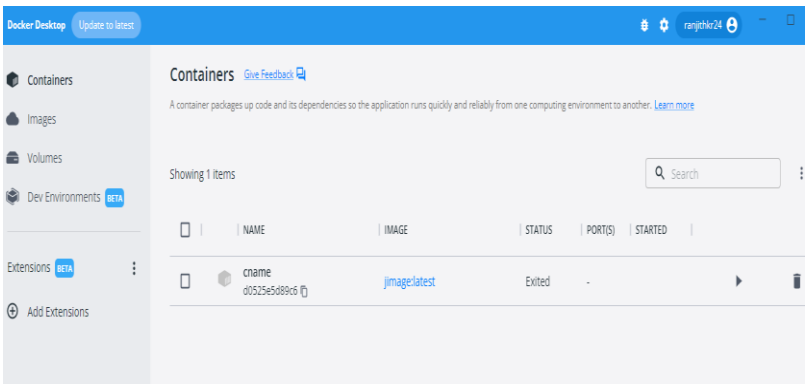
- **Verify the output**:

Run the following command to verify the output of the container:

$ docker logs mycontainer

This command displays the logs of the container and should show the programs output





Pushing images into the docker hub

Create a repository

Login to Dockerhub,Select Create Repository, name it and choose it to make itas public or private

Tag the Imageuse the command

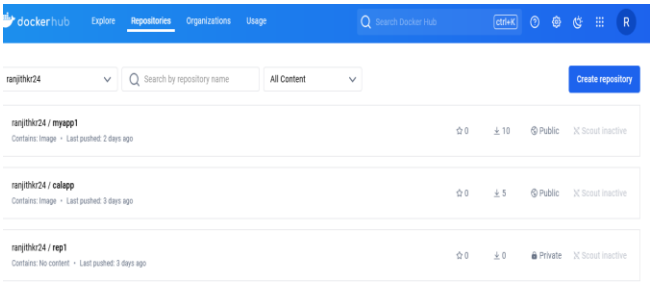
docker tag <image name><dockerhub-user name>/<repo-name><version>

Log into Docker

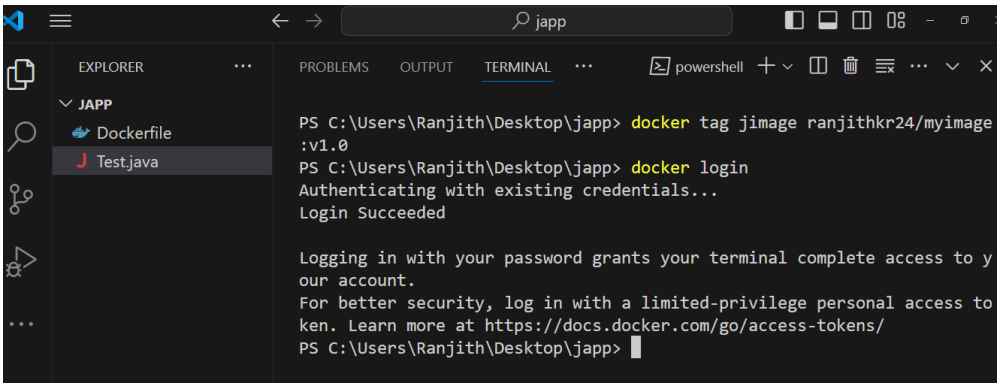
docker login **give login credentials**

Push the Imageuse the command **docker push <docker-hub-username>/<image name><version name>**

Login into Docker hub



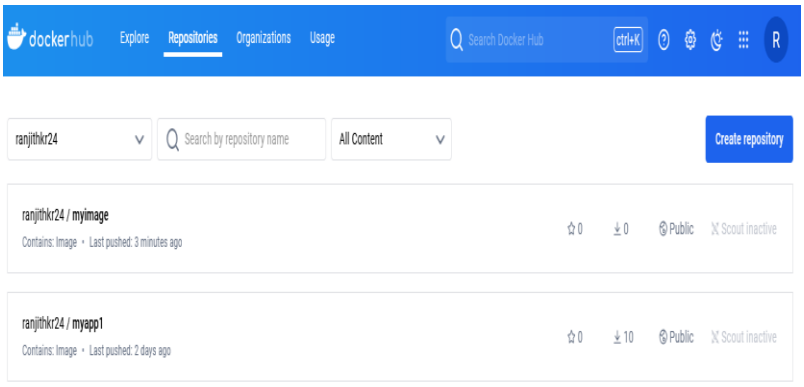
Tagging docker image and login into docker hub



Issuing docker push command to push the image

```
Logging in with your password grants your terminal complete access to y
our account.
For better security, log in with a limited-privilege personal access to
ken. Learn more at https://docs.docker.com/go/access-tokens/
PS C:\Users\Ranjith\Desktop\japp> docker push ranjithkr24/myimage:v1.0
The push refers to repository [docker.io/ranjithkr24/myimage]
d9081b57f2d9: Pushed
7f43ae0d9720: Pushed
020c055d83f3: Mounted from ranjithkr24/myapp1
56285d9a7760: Mounted from ranjithkr24/myapp1
077bff59ce57: Mounted from ranjithkr24/myapp1
9cd9df9ffc97: Mounted from ranjithkr24/myapp1
v1.0: digest: sha256:3d3de30bf1a86b342f65536c3122ff6552c015beaeb171e763
7d2d6d2342634f size: 1575
```

Pushed image can be viewed in dockerhub



Week 8: Integrate Kubernetes with Docker

Aim:

To enable Kubernetes on Docker Desktop and prepare system to run Kubernetes commands.

Kubernetes

Kubernetes, also known as K8s, is an **open-source system for automatingdeployment, scaling, and management of container-based applications** in different kinds of environments like physical, virtual, and cloud-native computing foundations.

Some of the benefits of Kubernetes

 Container Orchestration: Kubernetes manages the placement and scheduling of containers across a cluster of machines, ensuring that the desired state of the application is maintained.

 Scalability: Kubernetes allows applications to scale horizontally by automatically adding or removing containers based on resource utilization and user-defined rules.

 Self-Healing: Kubernetes monitors the health of containers. It automatically restarts failed containers or replaces them with new ones to ensure the desired state of the application is maintained.

 Rolling Updates and Rollbacks: Kubernetes supports rolling updates, allowing applications to be updated with minimal downtime. If any issues arise, it also facilitates rollbacks to previous versions.

 Storage Orchestration: Kubernetes provides a way to manage persistent storage volumes and attach them to containers as needed.

Differences between Docker and Kubernetes

Docker is used for **building, shipping, and running containers**—it provides the means to create isolated environments for applications.

On the other hand, Kubernetes focuses on orchestrating containers, meaning it helps **manage, scale, and ensure the smooth operation of large collections of containers**.